ta Science section, or don't have a Group Product Designer:

 create or update an issue with the problem to solve
 add the labels: UX and UX-AI (why the extra label: so we can easily have one board for all issues)
 tell us about the issue
   if the need is in the current milestone: link to your issue in Slack ai-ux-integration
   if the need is in the next 1-N milestones, add a milestone to the issue and let us know in the comments why that is the priority

If you have a group designer, plan for the Product Designer and UX Researcher assigned to your group to be the primary collaborators. Here is a suggestion for how to engage the AI UX team:

 Follow your normal process for creating an issue and prioritize the work with the Group Product Designer.
 Next, the Group Product Designer should review the work and note areas where they anticipate needing AI Framework support and collaboration. Mention the AI-UX group in an issue comment, along with the timeline for the work and specific or general areas they would like support.
 Alternatively, product designers can attend our office hours to bring questions or work in progress for review.

 How we prioritize

We do not currently have a large enough UX team to support all the work available, so we work with Product to set priorities.

 Product Leadership and the Product Design Manager will coordinate priorities.
 To see our current priorities, TBD
 The full scope of AI related UX work is tracked by milestone or by workflow label.
 As the team is currently small, we have a lightweight prioritization process that is primarily async. As we grow to more designers, we will likely formalize this with a scheduled review of the board.
 We aim to reply to UX asks within one week if it's for a future milestone, and two days if it's the current milestone.

SECTION: product/categories/_index.md

{{% include "includes/product-handbook-links.md" %}}

 Interfaces

We want intuitive interfaces both within the company and with the wider community. This makes it more efficient for everyone to contribute or to get a question answered. Therefore, the following interfaces are based on the product categories defined on this page:

- Home page
- Product page

- Product Features
- Pricing page
- DevOps Lifecycle
- DevOps Tools
- Product Direction
- Stage visions
- Documentation
- Engineering Engineering Manager/Developer/Designer titles, their expertise, and department, and team names.
- Product manager responsibilities which are detailed on this page
- Our pitch deck, the slides that we use to describe the company
- Strategic marketing specializations

Hierarchy

The categories form a hierarchy:

1. Sections: Are a collection of stages. We attempt to align these logically along common workflows like Dev, Sec and Ops.
Sections are maintained in data/sections.yml.
1. Stages: are maintained in data/stages.yml.
Each stage has a corresponding devops::<stage> label under the gitlab-org group.
1. Group: A stage has one or more groups.
Groups are maintained in data/stages.yml.
Each group has a corresponding group::<group> label under the gitlab-org group.
1. Categories: A group has one or more categories. Categories are high-level capabilities that may be a standalone product at another company. e.g. Portfolio Management. To the extent possible we should map categories to vendor categories defined by analysts.
There are a maximum of 8 high-level categories per stage to ensure we can display this on our website and pitch deck.
(Categories that do not show up on marketing pages show up here in italics and do not count toward this limit.) There may need to be fewer categories, or shorter category names, if the aggregate number of lines when rendered would exceed 13 lines, when accounting for category names to word-wrap, which occurs at approximately 15 characters.
Categories are maintained in data/categories.yml.
Each category has a corresponding Category:<Category> label under the gitlab-org group.
Category maturity is managed in the product Category Maturity Change process
1. Features: Small, discrete functionalities. e.g. Issue weights. Some common features are listed within parentheses to facilitate finding responsible PMs by keyword.
Features are maintained in data/features.yml.
It's recommended to associate feature labels to a category or a group with featurelabels in data/categories.yml or data/stages.yml.

Notes:

- Groups may have scope as large as all categories in a stage, or as small as a single category within a stage, but most will form part of a stage and have a few categories in them.

- Stage, group, category, and feature labels are used by the automated triage operation "Stage and group labels inference from category labels".
- We don't move categories based on capacity. We put the categories in the stages where they logically fit, from a customer perspective. If something is important and the right group doesn't have capacity for it, we adjust the hiring plan for that group, or do global optimizations to get there faster.
- We don't have silos. If one group needs something in a category that is owned by another group, go ahead and contribute it.
- This hierarchy includes both paid and unpaid features.

Naming

Anytime one hierarchy level's scope is the same as the one above or below it, they can share the same name.

For groups that have two or more categories, but not all categories in a stage, the group name must be a unique word or a summation of the categories they cover.

If you want to refer to a group in context of their stage you can write that as "Stage:Group". This can be useful in email signatures, job titles, and other communications. E.g. "Monitor:Health" rather than "Monitor Health" or "Monitor, Health."

When naming a new stage, group, or category, you should search the handbook and main marketing website to look for other naming conflicts which could confuse customers or employees. Uniqueness is preferred if possible to help drive clarity and reduce confusion. See additional product feature naming guidelines as well.

More Details

Every category listed on this page must have a link to a direction page. Categories may also have documentation and marketing page links. When linking to a category using the category name as the anchor text (e.g. from the chart on the homepage) you should use the URLs in the following hierarchy:

Marketing product page > docs page > direction page

E.g Link the marketing page. If there's no marketing page, link to the docs. If there's no docs, link to the direction page.

Solutions

Solutions can consist of multiple categories and are typically used to align to a customer challenge (e.g. the need to reduce security and compliance risk) or to market segments defined by analysts such as Software Composition Analysis (SCA). Solutions are also often used to align to challenges unique to an industry vertical (e.g. financial services), or to a sales segment (e.g. SMB vs Enterprise).

Solutions typically represent a customer challenge, and we define how GitLab capabilities come together to meet that challenge, with business benefits of using our solution.

Market segments defined by analysts don't always align to GitLab stages and categories and often include multiple categories. Two most frequently encountered are:

1. Software Composition Analysis (SCA) = Dependency Scanning + License Compliance + Container Scanning
1. Enterprise Agile Planning (EAP) = Team Planning + Planning Analytics + Portfolio Management + Requirements Management

We are intentional in not defining SCA as containing SAST and Code Quality despite some analysts using the term to also include those categories.

Capabilities

Capabilities can refer to stages, categories, or features, but not solutions.

Layers

Adding more layers to the hierarchy would give it more fidelity but would hurt usability in the following ways:

1. Harder to keep the interfaces up to date.
1. Harder to automatically update things.
1. Harder to train and test people.
1. Harder to display more levels.
1. Harder to reason, falsify, and talk about it.
1. Harder to define what level something should be in.
1. Harder to keep this page up to date.

We use this hierarchy to express our organizational structure within the Product and Engineering organizations.
Doing so serves the goals of:

- Making our groups externally recognizable as part of the DevOps lifecycle so that stakeholders can easily understand what teams might perform certain work
- Ensuring that internally we keep groups to a reasonable number of stable counterparts

As a result, it is considered an anti-pattern to how we've organized for categories to move between groups out
of concern for available capacity.

When designing the hierarchy, the number of sections should be kept small and only grow as the company needs to re-organize for span-of-control reasons. i.e. each section corresponds to a Director of Engineering and a Director of Product, so it's an expensive add. For stages, the DevOps loop stages should not be changed at all, as they're determined from an external source. At some point we may
change to a different established bucketing, or create our own, but that will involve a serious cross-functional conversation. While the additional value stages are our own construct, the loop and value stages combined are the primary stages we talk about in our marketing, sales, etc. and they shouldn't be changed

lightly. The other stages have more flexibility as they're not currently marketed in any way, however we should still strive to keep them as minimal as possible. Proliferation of a large number of stages makes the product surface area harder to reason about and communicate if/when we decide to market that surface area. As such, they're tied 1:1 with sections so they're the minimal number of stages that fit within our organizational structure. e.g. Growth was a single group under Enablement until we decided to add a Director layer for Growth; then it was promoted to a section with specialized groups under it. The various buckets under each of the non-DevOps stages are captured as different groups. Groups are also a non-marketing construct, so we expand the number of groups as needed for organizational purposes. Each group usually corresponds to a backend engineering manager and a product manager, so it's also an expensive add and we don't create groups just for a cleaner hierarchy; it has to be justified from a span-of-control perspective or limits to what one product manager can handle.

Category Statuses

Categories can have varying level of investment and development work. There are four main investment statuses:

1. Accelerated - Top category for product strategy that has received additional investment in the next year
1. Sustained - Categories where new features will be added in the next year
1. Reduced - Categories where scope and ambition is decreased although, new features will still be added in the next year
1. Maintenance - Categories where no new features will added

Typically, product direction pages will transparently state the investment status of the category for the fiscal year based on annual product themes and investment levels.

Changes

The impact of changes to sections, stages and groups is felt across the company.

All new category creation needs to be specifically approved via our Opportunity Canvas review process. This is to avoid scope creep and breadth at the expense of depth and user experience.

Merge requests with changes to sections, stages and groups and significant changes to categories need to be created, approved, and/or merged by each of the below:

1. Chief Product Officer
1. PLT Leader relevant to the affected Section(s)
1. The Director of Product relevant to the affected Section(s)
1. The Director of Engineering relevant to the affected Section(s)
1. Director of Product Design

Note: Chief Product Officer approval should be requested once all other approvals have been completed. To request approval, post the MR link in the chief-product-officer channel tagging

both the Chief Product Offcer and cc'ing the EBA to the Chief Product Officer.

The following people need to be on the merge request so they stay informed:

1. Chief Technology Officer
1. Development Leader relevant to the affected Section(s)
1. VP of Infrastructure & Quality Engineering
1. VP of UX
1. Director, Technical Writing
1. Engineering Productivity (by @ mentioning @gl-quality/eng-prod)
1. The Product Marketing Manager relevant to the stage group(s)

After approval and prior to merging, ping the Engineering Manager for Quality Engineering in the MR, if there are changes that:

- Add a new category, group, stage or section
- Move an existing category to a new or existing group
- Move an existing group to a new or existing stage
- Move an existing stage to a new or existing section
- Rename a group, stage or section
- Delete a group, stage or section

This is to ensure that GitLab Bot auto-labeling can be updated prior to the change, which can be disruptive if missed.

Upon approval, tag the group Technical Writer in the merge request to ensure documentation metadata is updated after the category change is merged.

Ensure that relevant slack channels are updated following our slack channel naming convention, open an access request to have slack channel names updated as they can no longer be updated by creators.

Examples

Because it helps to be specific about what is a significant change and what should trigger the above
approval process, below are non-exhaustive lists of examples that would and would not, respectively, require full approvals as outlined above.

Changes that require the above approvers include:

- Changes to a section, stage, group, or category name or marketing attribute
- Removal or addition of a section, stage, group, or category

Changes that require approval only from the relevant Product Leadership Team member include:

- Changing name or removing a non-marketing category, per the marketing attribute.

Changes that require approval only from the relevant Product Director include:

- Changing a category maturity date
- Changes to section or group member lists
- Changes to a category vision page

Changing group name

When changing the name of a group, create a merge request to change the group name in data/stages.yml
using the Group-Stage-Category-Change template,
and make sure to complete all the steps in the template.

When changing the team tags, such as beteamtag, ensure that each team member's individual data/teammembers/person/ YAML has the relevant departments entry updated. Alternatively, if the team tag is missing, add the tag under the list of departments as the second or lower entry.
The first departments entry is controlled by the Workday sync and will be overwritten.

When deciding on the naming, ensure that each team tag is unique. For example, sreteamtag should have a different value compared to beteamtag. If they are the same, then all team members with the tag with be displayed, duplicating the list for BE and SRE.

Changing category name

When changing an existing category name, there are some considerations to the order of events:

- First, create a MR to change the name in data/stages.yml and spec/homepage/categoryspec.rb.
- Get sign off from all required stakeholders listed in the instructions above.
- Merge the name change MR.
- Tag the category's technical writer so that they can update the documentation metadata
- Re-name the category direction page with a new MR. Search for the old category name on the category direction page to ensure the name has been updated in all places.
- Use the it-help Slack channel to request an update to Slack channel name for the re-named category

Changing category maturity

We primarily use the Category Maturity Scorecard process to determine category maturity.

Typically, category maturity moves up from planned to minimal to viable to complete to lovable. However, maturity can also be downgraded. For example, in cases where we discover a JTBD is not met (see example), or when we change the definition of maturity, we may choose to move category maturity down.

When downgrading product maturity, we adjust our customer's current expectations about our product. This is particularly impactful to our go-to-market team members in customer success and product marketing. We need to do the following to enable alignment between all affected and interested parties:

- Raise an MR for the downgrade and clearly state the reasons for change in the description (see example)
- Inform VP, Product Management by adding them as Reviewer on the MR

- Notify the product group counterparts in the MR; Product Marketing, Product Designer, Engineering Manager, and Technical Writer
- Post the MR in the customer-success slack channel prior to merging, to allow the team to assess impact and adjust
- Post the MR in the product slack channel for awareness

DevOps Stages

{{% product/categories %}}

Possible future Stages

We have boundless ambition, and we expect GitLab to continue to add new stages to the DevOps lifecycle. Below is a list of future stages we are considering:

1. Data, maybe leveraging Meltano product
1. Networking, maybe leveraging some of the open source standards for networking and/or Terraform networking providers
1. Design, we already have design management today

Stages are different from the application types you can service with GitLab.

Maturity

Not all categories are at the same level of maturity. Some are just minimal and some are lovable. See the category maturity page to see where each category stands.

Other functionality

This list of other functionality so you can easily find the team that owns it.
Maybe we should make our features easier to search to replace the section below.

Other functionality in Plan stage

Plan stage

Project Management group

Project Management group

- assignees
- milestones
- due dates
- labels
- issue weights
- quick actions
- email notifications

- to-do list
- Real-time features

Knowledge group

Knowledge group

- markdown functionality
- rich text editor

Other functionality in Create stage

Create stage

Code Review group

Code Review group

- Merge Requests
- GitLab CLI

Remote Development group

Remote Development group

- GitLab Workflow extension for Visual Studio Code

Other functionality in Verify

CI Group

CI Group

- CI Abuse Response

Pipeline Authoring Group

Pipeline Authoring Group

- CI/CD Template Management and Contributions

Other functionality in Monitor stage

Monitor stage

Other functionality in Engineering Productivity

Engineering Productivity

- GDK

Other functionality in Developer Experience

Developer Experience

- Reference Architectures
- GitLab Environment Toolkit (GET)
- GitLab Performance Tool (GPT)
- Performance Test Data
- Zero Downtime Testing Tool

Internal Customers: Gitaly, SaaS Platforms section, Infrastructure Department, Support Department, Customer Success

Other functionality in Analytics

Analytics

Product Analytics group

Product Analytics group

- Analytics Dashboards - used by many groups to add visualizations or provide pre-configured dashboards to users

Facilitated functionality

Some product areas are have a broad impact across multiple stages. Examples of this include, among others:

- Shared project views, like the project overview and settings page.
- Functionality specific to the admin area and not tied to a feature belonging to a particular stage.
- UI components available through our design system, Pajamas.
- Dashboards for displaying analytics, such as Product Analytics, Value Stream Analytics, and others.

While the mental models for these areas are maintained by specific stage groups, everyone is encouraged to contribute within the guidelines that those teams establish. For example, anyone can contribute a new setting following the established guidelines for Settings. When a contribution is submitted that does not conform to those guidelines, we merge it and "fix forward" to encourage innovation.

If you encounter an issue falling into a facilitated area:

- For issues that relate to updating the guidelines, apply the group::category label for the facilitating group.
- For issues that relate to adding content related to a facilitated area, apply the group::category label for the most closely related group. For example, when adding a new setting related to Merge Requests, apply the group::source code label.

Shared responsibility functionality

There are certain product capabilities that are foundational in nature and affect or refer to horizontal components of the architecture that have an impact across functional groups and stages.

These capabilities may refer to "Facilitated Functionality" (see section above) where the mental models are owned by a particular group, while anyone can contribute. However, there may be others that will not have a clear owner because they don't fall squarely into any particular group's purview of product categories. Prime examples of this are issues related to the improvement or evolution of foundational components, frameworks and libraries that are used by several or all groups across the organization. Another example could be components created by special task groups in the past that have been since dissolved and that have not required continued development to justify the funding of a dedicated permanent group to maintain them.

Whatever the source of the functionality, rather than thinking of these components as "not having an owner", it is important to think of them as being owned by everyone through the lens of shared responsibility. "Shared responsibility" means that every group should be committed and responsible to contribute to their continued maintenance, improvement and innovation.

Contribution, in this context, may manifest in different ways:

- Triage by coordinating conversations with stakeholders from different functions and at different levels to find the right owner and/or set the right level of priority.
- Product feature scoping and UX design by fleshing out the details of implementation in requirements documents and/or mockups.
- Technical scoping and feasibility analysis for possible technical and architectural approaches to implementation
- Actual implementation and release activities

It does not mean, however, that a single group should necessarily be solely responsible for all of these activities. Multiple groups could end up collaborating in execution. This coordination however requires a careful triage of the shared responsibility issues in the issue tracker where a single DRI coordinates these activities.

For more information please review this section in the quality department handbook to learn more about a decentralized approach to triaging these types of issues.

Categories A-Z

<!-- To edit the content of the Categories index, see: https://gitlab.com/gitlab-com/www-gitlab-com/-/blob/master/data/stages.yml -->

{{< product/categories-index >}}

SECTION: product/categories/features.md

<!-- Looking to update this content? Look at https://gitlab.com/gitlab-com/www-gitlab-

com/-/blob/master/data/categories.yml -->
Features by Group

This page is meant to showcase the features by tier across GitLab's Product Hierarchy.

{{% product/categories-features %}}

SECTION: product/categories/lookup.md

{{< product/categories-lookup >}}

SECTION: product/categories/gitlab-the-product/_index.md

{{% include "includes/product-handbook-links.md" %}}

GitLab the Product

Single application

We believe that a single application for the DevOps lifecycle
based on convention over configuration offers a superior user experience. The
advantage can be quoted from the Wikipedia page for convention over
configuration:
"decrease the number of decisions that developers need to make, gaining
simplicity, and not necessarily losing flexibility". In GitLab you only have to
specify unconventional aspects of your workflow. The happy path is
frictionless from planning to monitoring.

We're doubling down on our product for concurrent DevOps which brings the entire
lifecycle into one application and lets everyone contribute. We are leaning into
what our customers have told us they love: our single application strategy, our
pace of iteration, and our deep focus on users.

Consider opportunities to take advantage of this unique attribute in early
iterations. Integrating features with different parts of the application can
increase the adoption of early iterations. Other advantages:

- A minimal viable feature that is well integrated can be more useful than a
sophisticated feature that is not integrated.

Although not every feature needs to be integrated with other parts of the
application, you should consider if there are unique or powerful benefits for
integrating the feature more deeply in the second or third iteration.

GitLab.com

GitLab.com runs GitLab Enterprise Edition.

To keep our code easy to maintain and to make sure everyone reaps the benefits of all our efforts, we will not separate GitLab.com codebase from the Enterprise Edition codebase.

To avoid complexity, GitLab.com tiers and GitLab self-managed tiers are named the same.

GitLab.com subscription scope and tiers

GitLab.com subscriptions work on a namespace basis, which can mean:

- personal namespace, e.g. JaneDoe/
- group namespace, e.g. gitlab-org/

This means that group-level features are only available on the group namespace.

Public projects get Ultimate for free.
Public groups do not get Ultimate for free. Because:

- This would add significant additional complexity to the way we structure our features, licenses and groups. We don't want to discourage public groups, yet it wouldn't be fair to have a public group with only private projects and for that group to still get all the benefits of Ultimate. That would make buying a license for GitLab.com almost entirely moot.
- All value of group level features is aimed at organisations, i.e. managers and up (see our stewardship). The aim with giving all features away for free is to enable and encourage open source projects. The benefit of group-level features for open source projects is significantly diminished, therefore.
- Alternative solutions are hard to understand, and hard to maintain.

Admittedly, this is complex and can be confusing for product managers when implementing features.
Ideas to simplify this are welcome (but note that making personal namespaces equal to groups is not one of
them, as that introduces other issues).

For more guidance on feature tiers and pricing, visit tiering guidance for features

Deprecations & Removals Policy

Definitions

See the terminology of deprecations.

Is this a breaking change?

By definition, a removal is a breaking change. With a few exceptions, if the answer is yes to any of the following questions, the change is to be considered a breaking change and should be avoided other than for critical business risk.

- Does this change require an action from the customer to ensure continuity of function? (For example, removing background upload for object storage meant users needed to migrate objects to a supported object storage provider)
- Does this change cause a disruption to a customer's workflows or tasks? (For example, removing support for the "WIP" prefix in MRs meant users would need to use the "Draft:" prefix instead)
- Does this change cause other parts of the product to fail? (For example, removing certificate-based cluster integration means users can no longer install additional applications via GitLab Managed Apps)

For special definitions of what constitutes a breaking change for our APIs, see:

- REST API breaking changes.
- GraphQL API breaking changes.

For Experiment or Beta features, please see Support for experiment, beta, and generally available features.

Exceptions for breaking changes

Introducing a breaking change in a minor release is against policy because it can disrupt our customers, however there are some rare exceptions:

- When GitLab establishes that delaying a breaking change would overall have a significantly more negative impact to customers compared to shipping it in a minor release.
- If an integrated service shuts down, the integration can be removed during a minor release.

In all cases, the PM or EM must follow the Request a Breaking Change process.

Deprecating and removing features

Deprecating and removing a feature needs to follow a specific process because it is important that we minimize disruption for our users. As you move through the process, use the language deprecated or removed to specify the current state of the feature that is going to be or has been removed.

Process for deprecating and removing a feature

Please follow the process outlined in the docs.

Breaking Change Windows on GitLab.com

We deploy changes to GitLab.com many times a day. Because they are part of a continuous delivery process, these changes, including breaking changes, are not as predictable for customers.

Starting from GitLab 17.0, we introduced fixed windows during which breaking changes are rolled out to GitLab.com. The fixed windows are set as the Monday, Tuesday and Wednesday of the three weeks preceding the major release date, typically following the X.11 release date. A detailed example of what this looks like can be found in our 17.0 introduction issue.

Breaking changes should only be enabled during the breaking change windows. This means that where breaking changes are behind feature flags, the changes (feature flag) should only be switched during one of these windows to ensure customers workflows are not impacted outside of these communicated periods.

Product Managers will be responsible for making sure that as part of the deprecations and removals process, where applicable the deprecation/removal is aligned with a publicly communicated Breaking Change Window.

In June of 2023, we changed the process so that all deprecations and removals are displayed on the Deprecations page.
The announcements are grouped by the milestone they will be removed in. The deprecation announcement date is listed below each individual item.

Syntax deprecation process

{{% include "includes/syntax-deprection.md" %}}

Naming features

Naming new features or renaming existing features is notoriously hard and sensitive to many opinions.

Factors in picking a name

- It should clearly express what the feature is, in order to avoid the AWS naming situation.
- It should follow usability heuristics when in doubt.
- It should be common in the industry.
- It should not overlap with any other existing concepts in GitLab.
- It should have as few words as possible (so people won't use a shortened name).
- If you remove words from the name, it is still unique (helps to give it as few words as possible).
- We should also give products descriptive, not distinctive, names, and use prepositions when referring to third-party products and services in names. See our Product Principles for more information.

Process

- It's highly recommended to start discussing this as early as possible.
- Seek a broad range of opinions and consider the arguments carefully.
- The PM responsible for the area involved should make the final decision and not delay the naming.
- Naming should definitely not be a blocker for a feature being released.
- Reaching consensus is not the goal and not a requirement for establishing a name.

Renaming

The bar for renaming existing features is extremely high, especially for long-time features with a lot of usage.
Some valid but not exclusive reasons are:

- New branding opportunities
- Reducing confusion as we introduce new adjacent features
- Reducing confusion as we re-factor existing features

When renaming features follow the process to rename a category starting with an MR to change the name in data/features.yml. Request review by the Product Director and Engineering Director of your Section for approval. Optionally, add a comment including your technical writer, engineering manager, product design manager, and product designer for transparency.

When renaming a feature other items to consider are updates to documentation, blogs, direction pages and competitive information.

Using What's New to communicate updates to users

What's New is a feature that is part of GitLab.com and Self-managed GitLab that is used to communicate highlights from each release. After each major release, a yaml file is published that contains 3-10 highlights from the release along with links to the relevant documentation to get started using them.

A small notification dot appears above the "?" icon, and when users click on "What's new" in the menu, a drawer containing the updates slides into view.

The goal of What's New is to make it easy for users to be aware of the most important changes in GitLab so we can help them stay up-to-date on all of our changes and feature updates.

Permissions in GitLab

Use this section as guidance for using existing features and developing new ones.

1. Guests are not active contributors in private projects. They can only see, and leave comments and issues.
1. Reporters are read-only contributors: they can't write to the repository, but can on issues.
1. Developers are direct contributors, and have access to everything to go from idea to production,
unless something has been explicitly restricted (e.g. through branch protection).
1. Maintainers are super-developers: they are able to push to master, deploy to production.
This role is often held by maintainers and engineering managers.
1. Admin-only features can only be found in /admin. Outside of that, admins are the same as the highest possible permission (owner).
1. Owners are essentially group-admins. They can give access to groups and have destructive capabilities.
1. Auditor cannot access /admin, group settings, and project settings. Auditor has read-only access to all other areas.

To keep the permissions system clear and consistent we should improve our roles to match common flows
instead of introducing more and more permission configurations on each resource, if at all possible.

For big instances with many users, having one role for creating projects, doing code review and

managing teams may be insufficient.
So, in the long term, we want our permission system to explicitly cover the next roles:

1. An owner. A role with destructive and workflow enforcing permissions.
1. A manager. A role to keep a project running, add new team members etc.
1. A higher development role to do code review, approve contributions, and other development related tasks.

All the above can be achieved by iteratively improving existing roles.

Documentation on permissions

Security Paradigm

You can now find our security paradigm on the Secure Strategy page.

Also see our Secure Team engineering handbook.

Statistics and performance data

Traditionally, applications only reveal valuable information about usage and performance to administrators. However, most GitLab instances only have a handful of admins and they might not sign in very often. This means interesting data is rarely seen, even though it can help to motivate teams to learn from other teams, identify issues or simply make people in the organisation aware of adoption.

To this end, performance data and usage statistics should be available to all users by default. It's equally important that this can be optionally restricted to admins-only, as laws in some countries require this, such as Germany.

Internationalisation

GitLab is developed in English, but supports the contribution of other languages.

GitLab will always default to English. We will not infer the language / location / nationality of the user and change the language based on that. You can't safely infer user preferences from their system settings either. Technical users are used to this, usually writing software in English, even though their language in the office is different.

Performance

Fast applications are better applications. Everything from the core user experience, to building integrations and using the API is better if every query is quick, and every page loads fast. When you're building new features, performance has to be top of mind.

We must strive to make every single page fast. That means it's not acceptable for new pages to add to performance debt. When they ship, they should be fast.

You must account for all cases, from someone with a single object, to thousands of objects.

Read the handbook page relating to performance of GitLab.com, and note the Speed Index target shown there
(read it thoroughly if you need a detailed overview of performance). Then:

- Make sure that new pages and interactions meet the Speed Index target.
- Existing pages should never be significantly slowed down by the introduction of new features or changes.
- Pages that load very slowly (even if only under certain conditions) should be sped up by prioritizing work on their performance, or changes that would lead to improved page load speeds (such as pagination, showing less data, etc).
- Any page that takes more than 4 seconds to load (speed index) should be considered too slow.
- Use the availability & performance priority labels
to communicate and prioritize issues relating to performance.

You must prioritize improvements according to their impact (per the availability & performance priority labels).
Pages that are visited often should be prioritized over pages that rarely have any visitors.
However, if page load time approaches 4 seconds or more, they are considered no longer usable and should be fixed at the earliest opportunity.

Restriction of closed source JavaScript

In addition, to meet the ethical criteria of GNU,
all our JavaScript code on GitLab.com has to be free as in freedom.
Read more about this on GNU's website.

Why cycle time is important

The ability to monitor, visualize and improve upon cycle time (or: time to value) is fundamental to GitLab's product. A shorter cycle time will allow you to:

- respond to changing needs faster (i.e. skate to where the puck is going to be)
- ship smaller changes
- manage regressions, rollbacks, bugs better, because you're shipping smaller changes
- make more accurate predictions
- focus on improving customer experience, because you're able to respond to their needs faster

When we're adding new capabilities to GitLab, we tend to focus on things that will reduce the cycle time for our customers. This is why we choose
convention over configuration
and why we focus on automating the entire software development lifecycle.

All friction of setting up a new project and building the pipeline of tools you need to ship any kind of software should disappear when using GitLab.

Plays well with others

We understand that not everyone will use GitLab for everything all the time, especially when first adopting GitLab. We want you to use more of GitLab because you love that part of GitLab. GitLab plays well with others, even when you use only one part of GitLab it should be a great experience.

GitLab ships with built-in integrations to many popular applications. We aspire to have the world's best integrations for Slack, JIRA, and Jenkins.

Many other applications integrate with GitLab, and we are open to adding new integrations to our technology partners page. New integrations with GitLab can vary in richness and complexity; from a simple webhook, and all the way to a Project Service.

GitLab welcomes and supports new integrations to be created to extend collaborations with other products.
GitLab plays well with others by providing APIs for nearly anything you can do within GitLab.
GitLab can be a provider of authentication for external applications.

There is some natural tension between GitLab being a single-application for the entire DevOps lifecycle, and our support for better user experience via integration with existing DevOps tools. We'll prioritize first our efforts to improve the single-application
experience, second to enable a rich ecosystem of partners, and third to improve integration with the broader ecosystem
to other tools. GitLab is open-source so this should not prohibit contributors adding integrations for anything that they are missing - as long as it fits with GitLab product vision.

If you don't have time to contribute and are a customer we'll gladly work with you to design the API addition or integration you need.

SECTION: product/categories/gitlab-the-product/single-application/_index.md

Single application

GitLab is a complete DevOps platform, delivered as a single application that does everything from project planning and source code management to CI/CD, monitoring, and security. The advantages of a single application are listed in the following paragraphs.

By delivering a single application we shorten cycle times, increase productivity, and thus create value for our customers.

Other vendors offer a kit plane you have to assemble yourself, GitLab is a type certified aircraft.

Single application vs multiple applications

For example, the experience of one enterprise customer that converted from multiple DevOps tools to GitLab was:

- Nine-times fewer hours of elapsed time from when a developer chooses to work on a change to when it is in production
- Ten-times fewer separate tools for purchasing to buy, for IT to install, and for users to have to authenticate to
- Four-times fewer hours of hands-on keyboard time and four-times fewer tasks for people to do, allowing them to be more productive
- Five-times fewer different teams requiring to be involved and four-times fewer handoffs between teams, allowing them to be more productive and making the time to value more predictable

How does having one application vs many applications impact the workflow?

Dataflow and source data

Single authentication

You only have to login to one application. Users don't waste time logging in. You only have to set up one account. It is clear to users what account they should use. Due to less confusion phishing is harder.

Single authorization

GitLab does not require you to manage authorizations for each of its tools. This means that you set permissions once and everyone in your organization has the right access to every component.

Single project

Users don't have to set up or ask others to set up a project across applications. This can be a major source of delays.

Single setup

With Auto DevOps your new projects have all the needed testing and deployment from the get go. And by having logging and security scanning happen automatically you reduce risk.

Single interface

A single interface for all tools means that GitLab can always present the relevant context. Relatedly, any feature is always in its most optimal location. We don't have to force ourselves to create a new page to serve a new feature. The feature will just appear in the best designed existing location, simplifying navigation.

Also, you don't lose information due to constant context switching between different interfaces.

Users don't have to switch between applications constantly. Furthermore, if you're comfortable with one part of GitLab, you're comfortable

with all of GitLab, as it all builds on the same interface components.

Single installation

Running GitLab means that there is only one single application to install, maintain, scale, backup, network, and secure.

Single upgrade

Updating GitLab means that everything is guaranteed to work as it did before. Maintaining separate components is often complicated by upgrades that change or break integration points, essentially breaking your software delivery pipeline. This will never happen with GitLab because everything is tested as an integrated whole.

Single data-store

GitLab uses a single data-store so you can get information about the whole software development lifecycle instead of parts of it. With multiple applications you not only have multiple databases but also different definitions, processes. Multiple data-stores lead to redundant and inconsistent data. You don't have to duplicate data nor do manual entry. There is a single source of truth without having to build a data warehouse.

Single overview

Automated links between environments, code, issues, and epics provide a better overview of project state and progress. All information is realtime and there is a coherent set of concepts and definitions.

Single flow

Using a single application means you don't have to integrate 10 different products. This saves time from your developers, reduces risk, increases reliability, and lowers the bill of external integrators. The flow is better for the more than 100,000 organizations and over 2,000 people using GitLab that use the same flow to contribute and make it great. If you have a developer tools department, they can now focus on other tasks to make your developers more effective.

Single vendor

You can deal with one vendor instead of multiple vendors pointing to each-other.

Single training

Your end user training becomes less complex; a multi-vendor environment means multiple trainers to manage.

Single codebase

We prefer to offer a single application instead of a network of services or offering plugins for the following reasons:

1. We think a single application provides a better user experience than a modular approach, as detailed by this article from Stratechery.
1. The open source nature of GitLab ensures that we can combine great open source products.
1. Everyone can contribute to create a feature set that is more complete than other tools.
We'll focus on making all the parts work well together to create a better user experience.
1. Because GitLab is open source, the enhancements can become part of the codebase instead of being external. This ensures the automated tests for all
functionality are continually run, ensuring that additions keep working. This is in contrast to externally maintained plugins that might not be updated.
1. Having the enhancements as part of the codebase also
ensures GitLab can continue to evolve with its additions instead of being bound
to an API that is hard to change and that resists refactoring. Refactoring is essential to maintaining a codebase that is easy to contribute to.
1. Many people use GitLab on-premises, and in such situations it is much easier to install one tool than install and integrate many tools.
1. GitLab is used by many large organizations with complex purchasing processes, and having to buy only one subscription simplifies their purchasing.

Emergent benefits of a single application

A single application across the entire DevOps lifecycle has unique, emergent benefits.

- It's no longer necessary to ask for access to each separate tool; everyone is able to make use of all tools. Expect non-engineers to monitor deploys, follow the development process, and directly contribute to QA by reporting findings.
- Vastly improved cycle time. Constant context switching, re-authentication and lack of information slows down teams immensely. It sounds obvious, but having everything you need available at all times makes for more efficient work.
- Tracking whether a change is being worked on, is live in an environment, or is blocked no longer requires detective work. It's available everywhere and accessible to everyone.

This is highlighted by analyst specified benefits of Value Stream Delivery Platforms, of which GitLab is considered a representative vendor.

Some additional example benefits include:

CI and container registry

Pushing from CI to the container registry: before GitLab, this requires you to create a project and user account in a separate registry - and then on each job, credentials have to be passed
between CI and the registry. With GitLab, none of this is necessary, because GitLab knows who you are
and what your authorizations are in that project.

Debugging issues

Because all decisions, code, changes and deploys happen in the same place and are linked to

issues and merge
requests, GitLab has a full audit log of every decision, change and deploy. So if anything goes wrong,
discovering the source of the issues is simple and doesn't require checking multiple apps, teams and logs.

Monitoring from merge request

You deploy a change by merging a merge request. Because GitLab has monitoring built-in, it's able to show
in the merge request that e.g. the error rate has increased. The responsible developer sees this with the
correct context and can start to solve the issue immediately.

No need to integrate multiple tools

Enterprises that use a complex toolchain often need 20 people to manage all the interconnections while a single person can do the same work to administer GitLab. Here is a list of the various integrations necessary between tools:

1. Issue tracking <=> Kanban boards, preferably they show the same issues.
1. Issue tracking <=> Version control, close issues when you merged code in your branch.
1. Issue tracking <=> Code review, the code review has a link to the issue it is related to.
1. Issue tracking <=> CD/Release automation, see which changes are implemented by which deploy / are live and where.
1. Issue tracking <=> Monitoring, link the initiative to the impact on metrics.
1. Kanban boards <=> Version control, close issues when you merged code in your branch.
1. Version control <=> Code review, the code review happens on a branch that is updated.
1. Version control <=> Continuous integration, run CI automatically on the default branch, see CI status per branch.
1. Version control <=> CD/Release automation, see whether a particular commit is live somewhere.
1. Version control <=> Security testing, see whether a commit is vulnerable / with out-of-date dependencies.
1. Code review <=> Continuous integration, see the test results in the code review screen.
1. Code review <=> Security testing, see the test results in the code review screen.
1. Code review <=> CD/Release automation, see and control pushing to new environments in the code review screen.
1. Code review <=> Monitoring, see the effect of a code change on the metrics.
1. Continuous integration <=> Security testing, run security testing as part of CI.
1. Continuous integration <=> Container registry, push the container that is built to the registry.
1. Continuous integration <=> CD/Release automation, deploy if green, or don't deploy when red.
1. Continuous integration <=> Configuration management, configure the testing.
1. Security testing <=> CD/Release automation, prevent insecure code from being deployed.
1. Security testing <=> Container registry, scan the container registry.
1. Container registry <=> CD/Release automation, pull the container.
1. CD/Release automation <=> Configuration management, configure the deployment.
1. CD/Release automation <=> Monitoring, see the release in the monitoring.
1. Configuration management <=> Monitoring, configure the monitoring.

Flowchart

Below is a flowchart illustration of the various integrations necessary between tools specified above.

```
mermaid
flowchart LR
  A(Plan)
  B(Create)
  C(Release)
  D(Software Supply Chain Security)
  E(Configure)
  F(Monitor)
  G(Verify)
  H(Secure)
  I(Package)

  A <--> B
  A <--> C
  A <--> F
  B <--> G
  B <--> C
  B <--> H
  B <--> F
  C <--> G
  C <--> F
  C <--> D
  E <--> C
  G <--> H
  G <--> I
  G <--> C
  G <--> F
  H <--> C
  H <--> I
  I <--> C

  subgraph Manage
    A
    B
    C
    D
    E
    F
    G
    H
    I
  end
```

Security benefits

When we have added aggregated logging we can move on to some security features that only a single application for the whole DevOps lifecycle can offer out of the box:

1. Log complete requests (header, payload, cookie) from Nginx to Elastic Search to form the basis of an Intrusion Detection System (IDS) and give you complete sitemaps. We can probably use something like clientbodyinfileonly and we don't need something like MITM proxy since we can use the SSL keys.
1. Log complete sessions, that contain multiple requests over time, such as logging in and creating a new object before the final request is possible.
1. Modify the payload or headers with something like wfuzz and content like seclists, run it against a review app and see what http status returns change from 2xx to 5xx and/or where you can do a SQL string escape. Show a sorted list of starting with the most likely vulnerabilities.
1. When running the above fuzzing payloads see if they hit new code paths by instrumenting the application with runtime application self protection and then sort by seeing what code paths are the least covered by the unit tests ran by GitLab CI.
1. Any error tracking report contains all the requests in that user session to generate that state.
1. When someone submits a vulnerability report combine this with any 500 error stack traces stored in error tracking.
1. When fuzzing or a human hits a vulnerability generate a pcap file with the request and start scanning production traffic for this.

Lower operating expense (OpEx)

Building and maintaining integration between multiple individual point tools comes with additional overt and
hidden costs.

Overt cost

The overt cost of paying for licensing and support of multiple tools is
higher than a single application. A single application can charge less because its fixed costs are distributed
across the functionality, whereas separate vendors each need to pay those costs themselves for each of their solutions.

Hidden cost

Businesses that manage a toolchain pay hidden operating costs in the form of engineering time needed to
build and maintain the toolchain instead of using those engineering resources to write software with
business logic that delivers differentiated value. For a large enterprise this can be the difference
between paying 20 engineers vs one engineer
to maintain your tools.

Further examples

1. Planning (code merge closes issues updates epic) - Always up to date and across different projects
1. Move from issue board to issue tracker - Kanban boards are consistent with issue tracker
1. Getting the stack running for a new developer - Can start developing without setup time or help
1. Merge request contains monitoring, security, etc. information - Can see overview in MR of tests, quality, performance, browser performance, artifacts, discussion, environment, rollback
1. Security (security in parallel, instead of separate) - Security tests results can be aggregated
1. Identify vulnerable applications quickly and automate the remedy
1. Add the build application to a container registry - No need to set up a project in the container registry and pass credentials around
1. Need to find an artifact from a CI build - Artifacts are accessible directly from any merge request
1. Reviewing the application - QA test can start work immediately
1. Run CI/CD/Security/Quality without having to configure anything - One click to deploy to production
1. Need to scale up a containerized application - Scaling up and down is just another button inside your project with all relevant data to make a decision
1. A project adopts containerization - New servers are automatically provisioned via Kubernetes
1. Rollback a failing deployment automatically without human intervention
1. Activity feeds span all product categories, across DevOps lifecycle
1. Cycle time (real time and integrated) - Can view activity across DevOps lifecycle
1. Permission levels and settings - User rights are consistent across all applications
1. Audit log - What a person was able to do across the lifecycle in audit logs
1. Geo does not just to SCM, it also does CI, CD, issue tracking - People at other location have quick access to the whole DevOps-lifecycle, not just repos
1. Easier to administer then 10 different applications - The flow between the activities doesn't break because of updates to a single step in the life-cycle
1. Start a new microservice project - A project has all tools in it, no need to find out how this project is named in other tools
1. Someone new joins the organization - Users automatically have access to all tools that are relevant for their role, without creating a ticket and waiting

Conversational Development

Conversational development carries a conversation across Departments and teams through the DevOps lifecycle, involving gatekeepers at every step. By providing relevant context, a feature that is only possible with an integrated solution like GitLab, we can reduce cycle time, making it easier to diagnose problems and make decisions.

Concretely, conversational development in GitLab means that a change can be easily followed from inception to the changes it made in performance and business metrics and feeding this information back to all stakeholders immediately.

Effectively, this allows cross-functional teams to collaborate effectively.

Review Apps

Review apps are the future of change review. They allow you to review not just the code, but the actual changes in a live environment. This means one no longer has to check out code locally to verify changes in a development environment, but you simply click on the environment link and try things out.

Only a tool that combines code review with CI/CD pipelines and integration with container schedulers (Kubernetes) is able to quickly create and shut down review apps and make them part of the review process.

Cycle Analytics

Cycle analytics tell you how what your time to value, from planning to monitoring is. Only by having direct access to each step in the software development lifecycle, GitLab can give actionable data on time to value.

This means you can see where you are lagging and make meaningful change to ship faster.

Container registry integrates with CI

Every GitLab projects comes with a container registry. That means there is no need for elaborate configuration to be able to use and push container images in CI. Rather, all you have to do is use a pre-defined variable in your CI configuration file (.gitlab-ci.yml).

Use cases, not modules

Sometimes users ask if GitLab has particular module that contains a set of features. A modules-mindset results in narrowly-focused solutions and redundant features and abstractions. GitLab is a single application that solves entire use cases that might touch many parts of GitLab, without creating additional unnecessary complexity.

Existing features and data accrue value over time

As GitLab breadth expands, but is maintained as a single application, new features are continually back-integrated into existing features. This means that existing features and the data that they have already generated over time accrue new value for very little cost in the context of the new features and their use cases.

Trend Towards DevOps Platform Consolidation

Continuing apace after Microsoft's 2018 acquisition of GitHub, the trend to consolidate DevOps companies seems here to stay. In January 2019, Travis CI was acquired by Idera, and in February 2019 we saw Shippable acquired by JFrog. Atlassian and GitHub now both bundle CI/CD with SCM, alongside their ever-growing related suite of products. In January 2020, CollabNet acquired XebiaLabs to build out their version of a comprehensive DevOps solution.

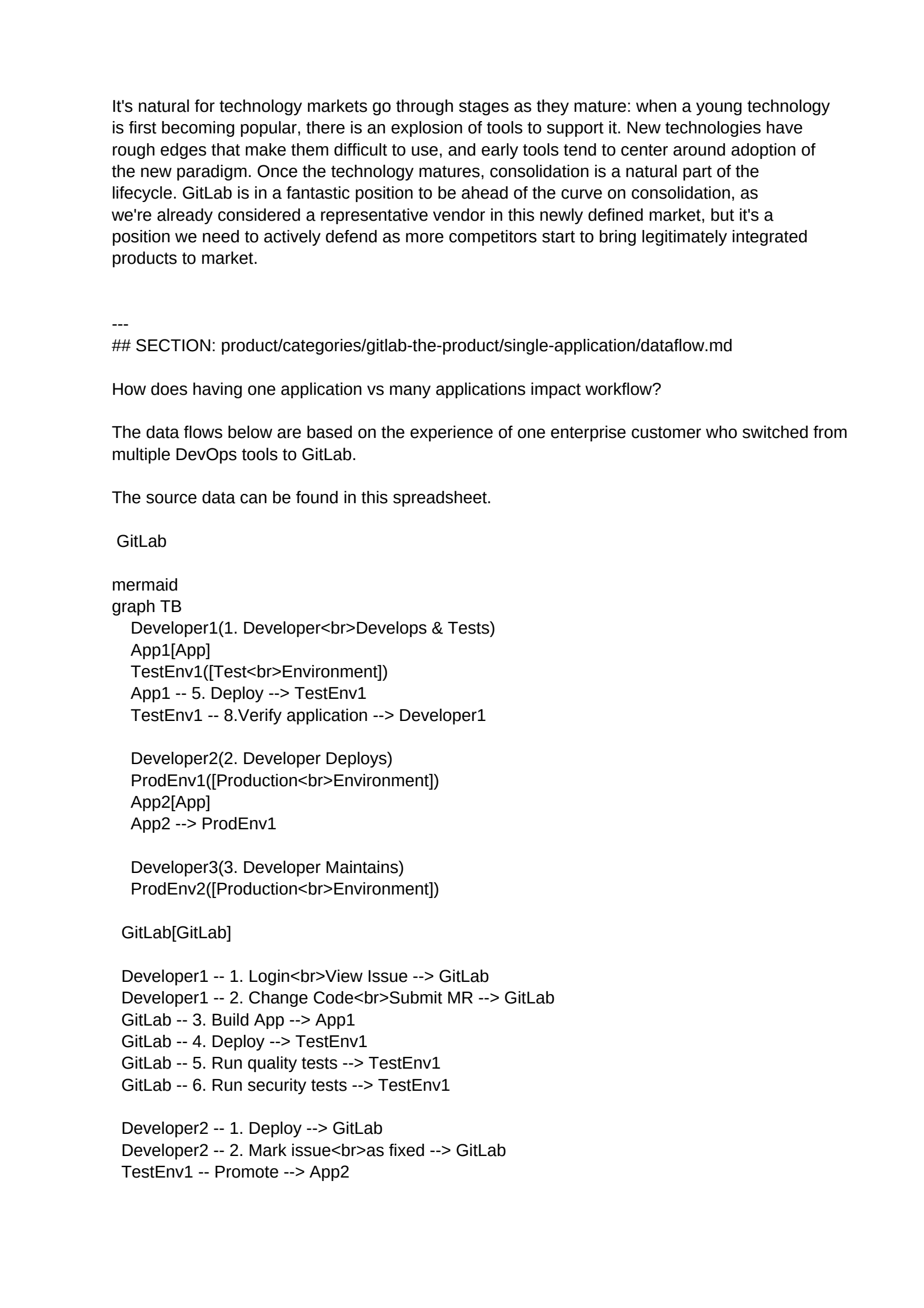
It's natural for technology markets go through stages as they mature: when a young technology is first becoming popular, there is an explosion of tools to support it. New technologies have rough edges that make them difficult to use, and early tools tend to center around adoption of the new paradigm. Once the technology matures, consolidation is a natural part of the lifecycle. GitLab is in a fantastic position to be ahead of the curve on consolidation, as we're already considered a representative vendor in this newly defined market, but it's a position we need to actively defend as more competitors start to bring legitimately integrated products to market.

SECTION: product/categories/gitlab-the-product/single-application/dataflow.md

How does having one application vs many applications impact workflow?

The data flows below are based on the experience of one enterprise customer who switched from multiple DevOps tools to GitLab.

The source data can be found in this spreadsheet.



```
  ProdEnv2 -- 1. Application Logs --> GitLab
  ProdEnv2 -- 2. Application Metrics --> GitLab
  Developer3 -- 3. Review Logs --> GitLab

  classDef default fill:FFFFFF,stroke:0C7CBA;
  %%class GitLab,Developer test


 Multiple DevOps Tools

 mermaid

graph TB

DeveloperMain(Developer)

    AppD(App)
    SourceControlD(Source Control)
    CIToolD(CI Tool)
    CDToolD(CD Tool)
    TestEnvD(Test Env)
    IssueTrackerD(Issue Tracker)
    DeveloperD(Developer)
    DeveloperD -- 1. Login --> IssueTrackerD
    DeveloperD -- 2. View Issue --> IssueTrackerD
    DeveloperD -- 3. Login --> SourceControlD
    DeveloperD -- 4. View Issue --> SourceControlD
    DeveloperD -- 5. Login --> CIToolD
    DeveloperD -- 6. Submit MR --> CIToolD
    SourceControlD --> CIToolD
    CIToolD -- 7. Build --> AppD
    DeveloperD -- 8. Login --> CDToolD
    DeveloperD -- 9. Deploy --> CDToolD
    CDToolD -- 11. Deploy --> TestEnvD
    AppD --10. Pull --> CDToolD
    TestEnvD -- 12. Verify --> DeveloperD

SecEngMain(Security Eng)

SecEngMain--> SecEngT
SecEngMain --> SecEngD

DeveloperMain --1. Develop -->DeveloperD
DeveloperMain --2. Test -->DeveloperT

  DeveloperT(Developer)
  TestToolT(Test Tool)
  SAST(SAST Tool)
  SecretScan(Secret Scan)
  DependencyScan(Dependency Scan)
```

```
  SecEngT(Security Eng)
  TestResults(Test Results)
  QualityTeamT(Quality Team)
  DeveloperT --1. Login & Run Tests --> TestToolT
  TestToolT --> TestResults
  SAST --> TestResults
  SecretScan --> TestResults
  DAST --> TestResults
  DependencyScan --> TestResults
  DeveloperT -- 2. Login & Run Tests --> SAST
  DeveloperT -- 3. Login & Run Tests --> SecretScan
  DeveloperT -- 4. Login & Run Tests --> DependencyScan
  DeveloperT -- 5. RequestDAST Scan--> SecEngT
  SecEngT -- 6.  Login & Run Tests --> DAST
  TestResults -- 7. Review Results --> DeveloperT
  TestToolT --> SAST
  SAST --> SecretScan
  SecretScan --> DependencyScan
  DependencyScan --> DAST
  TestToolT -- 8. Review results --> QualityTeamT

DeveloperMain --3. Deploy --> DeveloperDep

  DeveloperDep(Developer)
  QualityTeamDep(Quality Team)
  ProdOpsD(Production Ops)
  SecEngD(Security Eng)
  CDTool(CD Tool)
  ProdEnv(Prod Env)
  IssueTrackerDep(Issue Tracker)
  DeveloperDep -- 1. Request Approval --> QualityTeamDep
  QualityTeamDep --2. Approval --> SecEngD
  SecEngD -- 3. Approval --> ProdOpsD
  ProdOpsD -- 4. Login and Deploy --> CDTool
  CDTool --5. Deploy --> ProdEnv
  ProdOpsD --6. Complete --> DeveloperDep
  DeveloperDep -- 7.  Close issue --> IssueTrackerDep

DeveloperMain -- 4. Maintain -->DeveloperM
QualityTeamMain(QualityTeam)
QualityTeamMain --> QualityTeamT
QualityTeamMain --> QualityTeamDep
ProdOpsMain(Production Ops)
ProdOpsMain --> ProdOpsD
ProdOpsMain --> ProdOpsMaintain

  DeveloperM(Developer)
  ProdEnvM(Prod Env)
  LogApp(Log App)
  MetricsApp(Metrics App)
```

```
  ProdOpsMaintain(Production Ops)
  ProdEnvM --1 . Logs --> LogApp
  LogApp --2. Metrics --> MetricsApp
  DeveloperM -- 3. Login & View--> LogApp
  DeveloperM --4. Login & View --> MetricsApp
  ProdOpsMaintain -- 3. Login & View--> LogApp
  ProdOpsMaintain --4. Login & View --> MetricsApp

  classDef default fill:FFFFFF,stroke:0C7CBA;
  %%class GitLab,Developer test
```

SECTION: product/cross-stage-features/_index.md

Cross-stage feature collaboration

Each stage is responsible for building functionality that furthers their vision and direction. Even if some components of that functionality happen to cross into spaces typically owned by other stages, they should still build it (if it's important to them).

If the feature isn't necessary or urgently needed to move forward (for example, it won't block another feature's development), then you can always consider putting it on the backlog of the stage that owns that feature.

Here are some guidelines for thinking about "Which stage should do this work?":

1. If a stage wants to develop new functionality that is core to their value, even if it happens to live inside a feature owned by another stage, they should still build it.
1. Alternately, if the functionality lives inside another stage's feature, but is also very-much a "nice-to-have", they should consider putting it in an issue and labeling it appropriately. This way, the stage that owns that feature can prioritize it at a later date when it makes sense for them